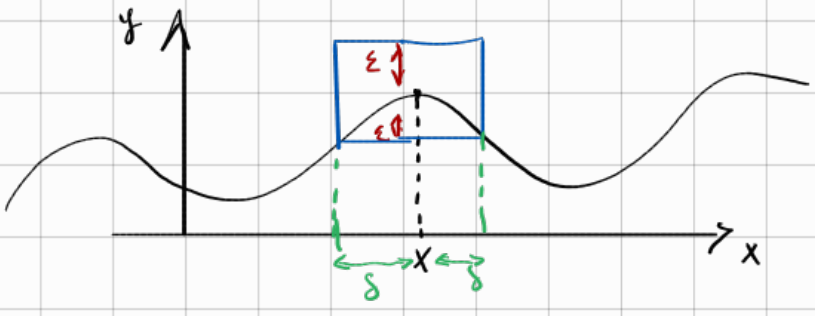
$f: \mathbb{R}^N \to \mathbb{R}$ è continua se:

$$\forall \underline{x} \in \mathbb{R}, \ \forall \varepsilon > 0, \ \forall \delta > 0 \ : \forall \underline{y} \in \mathbb{R}^N, \ \|x - y\| < \delta \Rightarrow \|f(\underline{x}) - f(\underline{y})\| < \varepsilon$$

Per $N = 1$



Continua separatamente $\begin{cases} \to x, \text{ con } y \text{ fissato} \\ \to y, \text{ con } x \text{ fissato} \end{cases}$ } anche se f è continua separatamente in x e y non è detto che sia continua in $\mathbb{R}$

Una $f: \mathbb{R}^N \to \mathbb{R}$ è derivabile in $\underline{x}_0$ lungo una direzione v, $\|v\| = 1$ se $\exists$ e finito il limite del rapporto incrementale in $\underline{x}_0$

$$\frac{\partial f}{\partial v}(\underline{x}_0) = \lim_{t \to 0} \frac{f(\underline{x}_0 + tv) - f(\underline{x}_0)}{t}$$

Derivata direzionale

Una derivata parziale di $f: \mathbb{R}^N \to \mathbb{R}$ è la derivata rispetto alla x_i-esima variabile, se il limite del rapp. incrementale

$$\frac{\partial f}{\partial x_i}(\underline{x}_0) = \lim_{t \to 0} \frac{f(x_1, x_2, \dots, x_i + t, x_{i+1}, \dots, x_n) - f(\underline{x}_0)}{t}$$

L'esistenza $\frac{df}{dx}$ in $\underline{x}_0$ garantisce:

Per $N = 1$:

- $\exists$ il limite del rapporto in x

- $\exists$ tg di grafico in $\underline{x}_0$
- f continua in $\underline{x}_0$

L'esistenza di $\frac{df}{dx_i}$ in x_0 non garantisce

Per $n > 1$:

- $\exists$ di un piano tg al grafico
- Continuità

Per poter definire il gradiente di f (∇f) in $\underline{x}_0$, devono poter esistere tutte le derivate parziali di f in $\underline{x}_0$

Si può definire $\nabla f(x_0) = \left(\frac{\partial f}{\partial x_1}(\underline{x}_0), \frac{\partial f}{\partial x_2}(\underline{x}_0), \dots, \frac{\partial f}{\partial x_n}(\underline{x}_0) \right)$

$\underline{e}_1$ direzione, $\underline{e}_2$, $\underline{e}_n$

Se esiste l'applicazione lineare in $\underline{x}_0 \in \mathbb{R}^n$ di direzione $\underline{v}$ detta DIFFERENZIALE

$$\forall \underline{v} \in \mathbb{R}^n : df(\underline{v}) = \underline{v} \cdot \nabla f(\underline{x}_0) = \frac{\partial f}{\partial \underline{v}}(\underline{x}_0)$$

Allora f è DIFFERENZIABILE $\longrightarrow$ Continua in $\underline{x}_0$

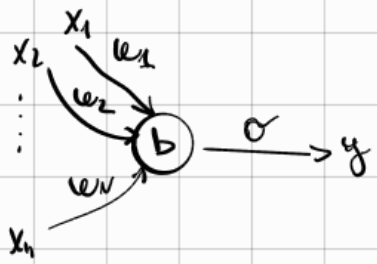
$\longrightarrow \exists$ il piano tg in x_0

Se $\underline{x}_0$ è un punto di max o min, $df(\underline{x}_0) = 0$

$\hookrightarrow \nabla f(\underline{x}_0) = 0$

$\hookrightarrow \nabla f$ indica la direzione di massima crescita

Perceptron / Neuron



$f: \mathbb{R}^N \rightarrow \mathbb{R}$

σ: funz. d'attivazione

$$f(\underline{x}) = \sigma(\underline{x} \cdot \underline{w} + b)$$

- SLP
  - Due layer
    - Input
    - Output
  - $G: \langle V, A \rangle$; completo, aciclico, bipartito e diretto
    - I vertici sono i neuroni
  - È fully connected
  - Parametri $\rightarrow \theta = (\underline{w}, \underline{b})$
    - Matrice dei pesi $W \in \mathbb{R}^{N \times M}$
    - Vettore di bias
  - $f: \mathbb{R}^N \rightarrow \mathbb{R}^M$
  - Limitazioni
    - Non in grado di approx qualsiasi funz. continua
    - Impossibilità a generalizzare su dataset non lineari (audio, testo, immagini)

- MLP
  - $G: \langle V, A \rangle$; completo, aciclico

MLP

→ L layer nascosti → Almeno un layer nascosto

↓ $L_{IN} \cup L_1 \cup \dots \cup L_L \cup L_{out}$

↳ 1 layer di input, 1 di output

$$F(\underline{x}) = \sigma(W^{(2)T} \cdot \sigma(W^{(1)T} \cdot \underline{x} + b^{(1)}) + b^{(2)})$$

↳ Esempio MLP con uno strato nascosto

Vantaggi rispetto all'SLP → Approx. tutte le funz. continue

↓ Teorema di approx. universale

↓ Backpropagation

↓ Risolve problemi non lineari

Addestramento supervisionato → Machine learning

$D_{train} = \{ (\underline{x}, \underline{y}) : \underline{x} \in \mathbb{R}^N, \ \underline{y} \in \mathbb{R}^N \}$

$D_{val} \cap D_{train} = \emptyset$

$D_{test} \cap D_{train} = \emptyset$

Loss: MSE → Penalizza di più gli errori > 1

$\mathcal{L} = \| F(\underline{x}) - \underline{f}(\underline{x}) \|^2$ (output modello, y dataset)

Geometria del bravo → L1 $\mathcal{L} = \| F(\underline{x}) - \underline{f}(\underline{x}) \|$ Calcola lo spostamento lungo gli assi

Fase di training su D_{train}

- Minimizzare la loss → discesa del gradiente
- Modificare pesi e bias della rete
- $\underline{\theta}_k = \underline{\theta}_{k-1} - \eta \nabla \mathcal{L}(\underline{\theta}_{k-1})$
- $\theta = (W, b)$

Finito il training si testa il modello su D_{test}

- Modello pronto → Fase di inferenza

Algoritmo di training

- Questo per ogni stato della rete
- Inizializzazione dei pesi e bias
- Passo forward, si calcola l'output
- Passo backward, propagazione dell'errore all'indietro

Parametri:

- η: learning rate
- N_{epoche}: num. epoche
- D_{train}
- Architettura rete: L (stati nascosti)

Aggiornamento pesi e bias
(A fine iterazione di training)

Eq. back propagation:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta_j^{(l)} \cdot \sigma(s_i^{(l-1)})$$

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta_j^{(l)}$$

} Calcolano i gradienti della loss

Modo efficiente per calcolare $\nabla \mathcal{L}$, grazie alla chain-rule possiamo calcolare $\frac{\partial \mathcal{L}}{\partial w_{ij}}$, $\frac{\partial \mathcal{L}}{\partial b_j}$

Immagine $I: \mathbb{R}^2 \to \mathcal{C}$

Campionamento

Trasforma l'immagine in una griglia di pixel $H \times W$

Quantizzazione

Indica il canale dei colori:
- $RGB \in \mathbb{R}^3$; Red Green, Blue
- $HSV \in \mathbb{R}^3$; Hue, Saturation, Value

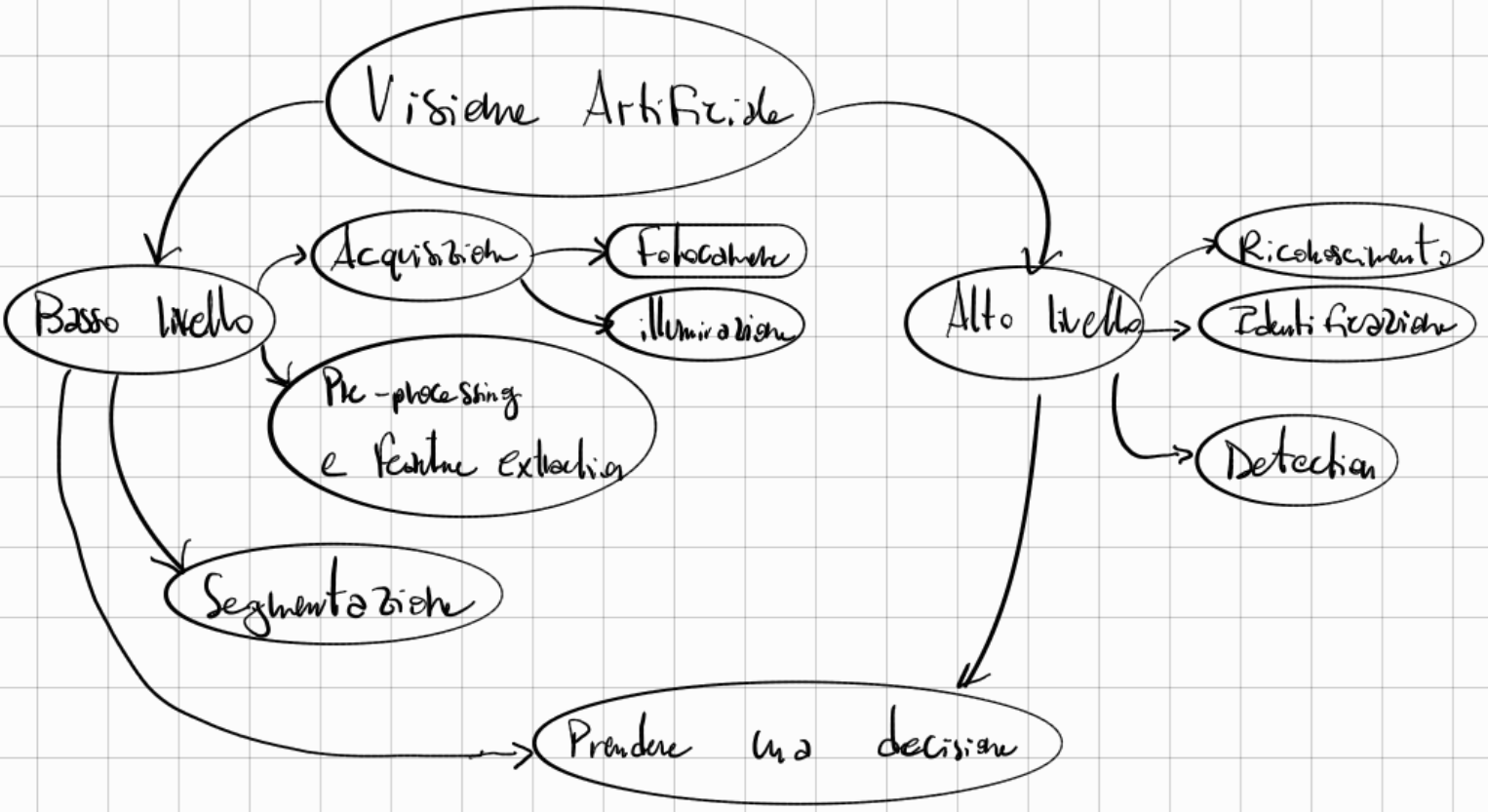
2 Parametri:

N_C: Num. di colori da rappresentare

N_B: Numero di bit

$2^{N_B} \geq N_C$

L'immagine digitale è un Tensore $I: \underbrace{C}_{\text{canale dei colori}} \times (H \times \mathbb{R}) \to \mathbb{R}$

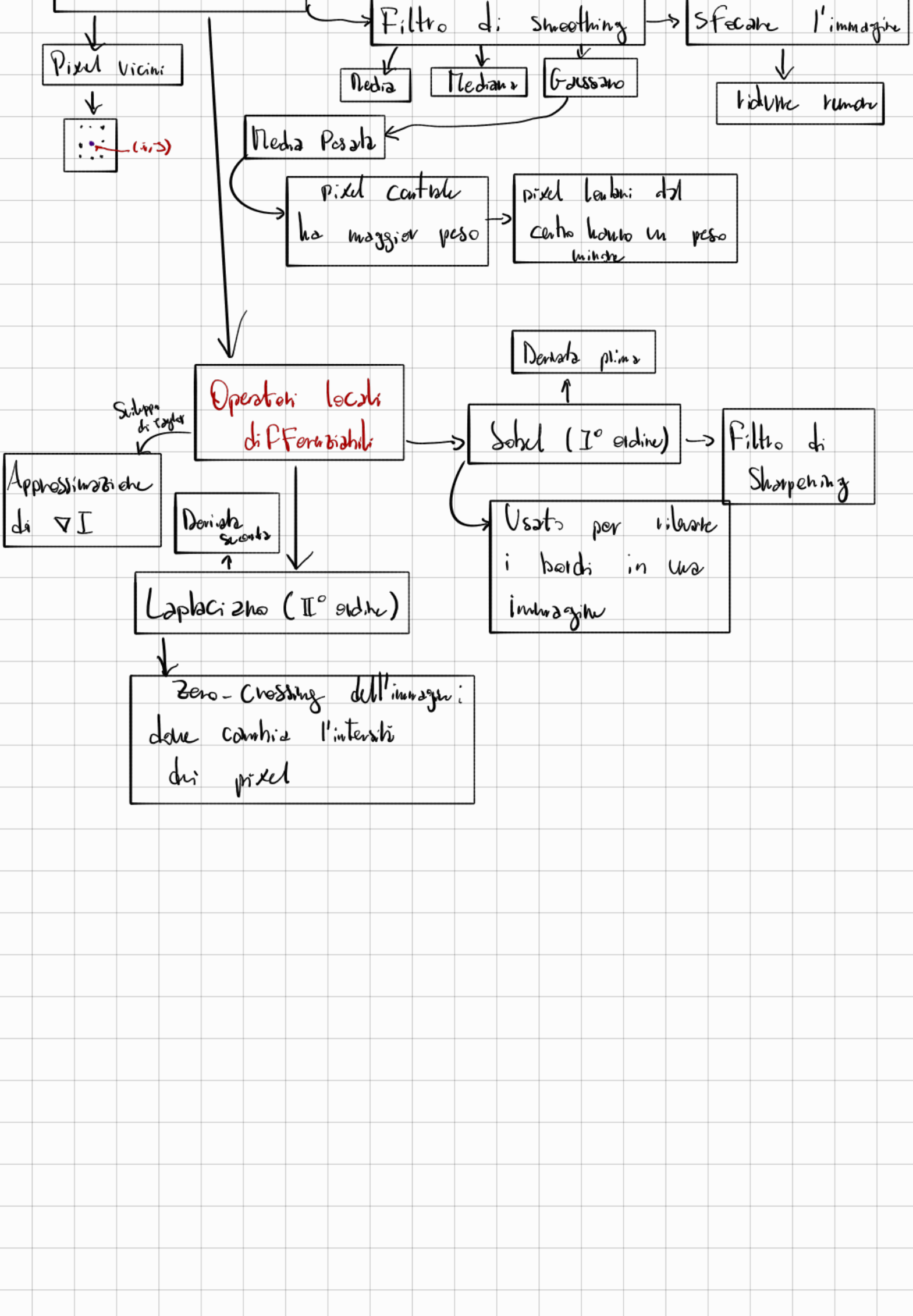


Visione di Basso livello

Operatori
- Puntuali, singolo pixel
- Locali, singolo pixel + vicinato
- Globali, tutti i pixel

Operatori Puntuali
- Istogramma livelli di grigio
  - $h = \{0, 1, 2, \dots, 2^{N-1}\} \to \mathbb{N}$
  - Misura la distribuzione dell'intensità di livelli di grigio
- Inversione livelli di grigio
- Compressione logaritmica
  - Espande i livelli bassi
  - Comprime i livelli alti
- Compressione potenza
- Espansione di contrasto

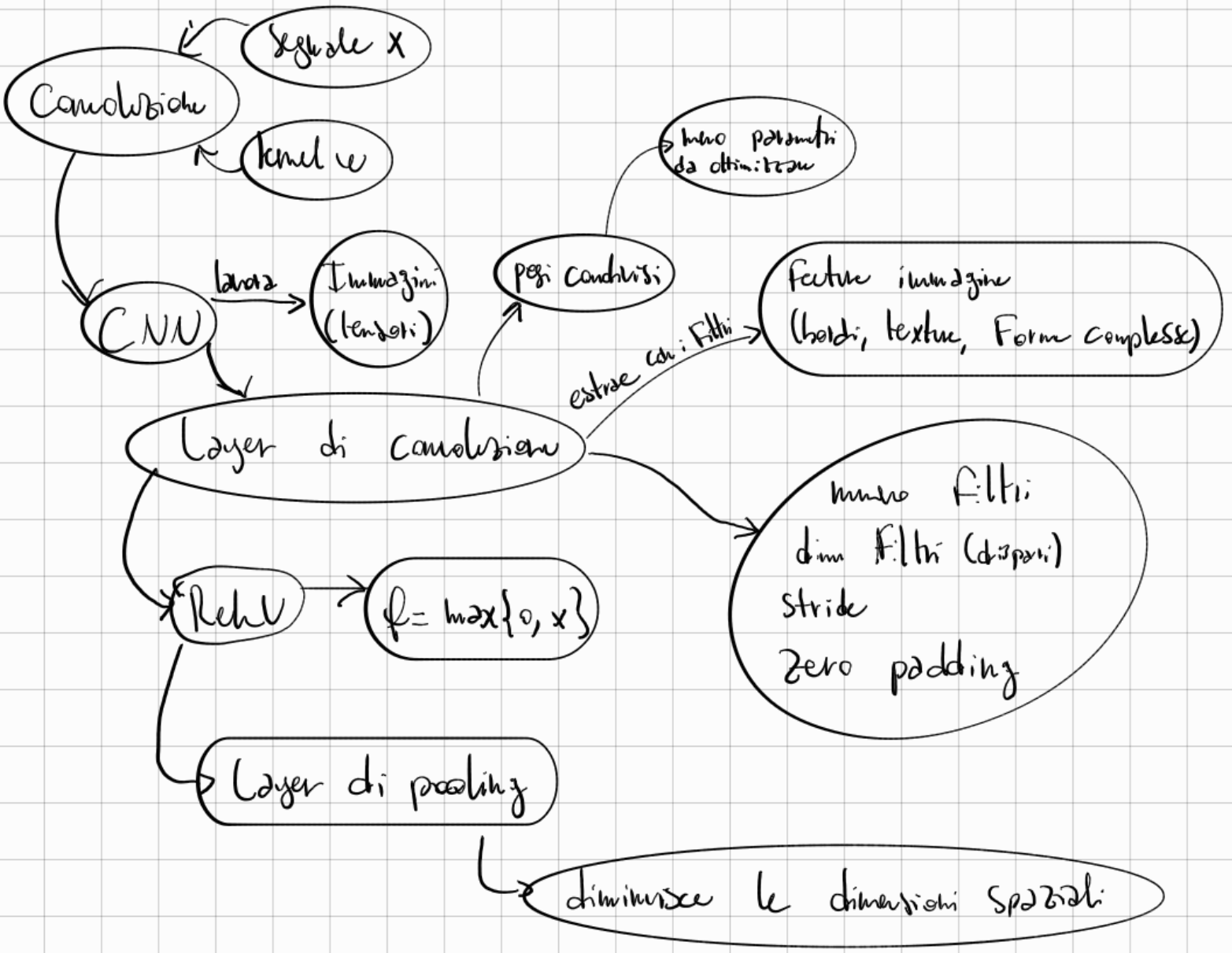
Operatori locali
- Filtro di sharpening
  - Lavora sui bordi
  - Dare nitidezza

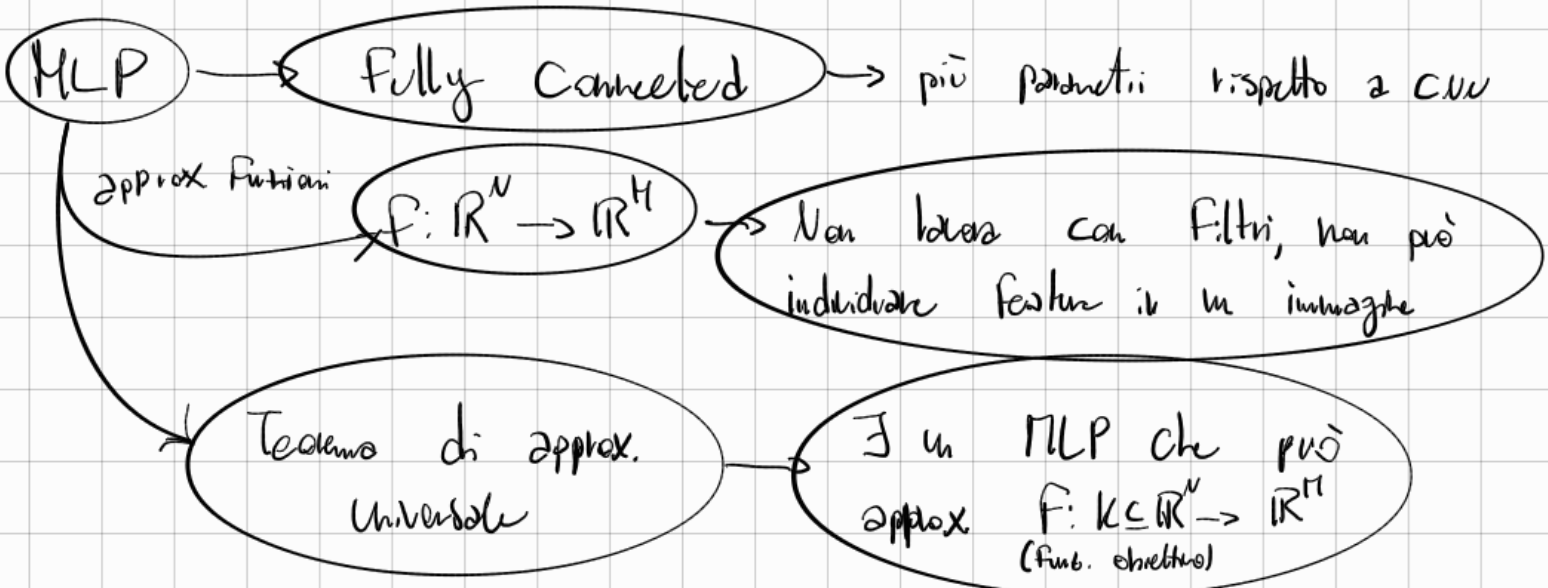

Filtro di Smoothing
Sfocare l'immagine
Pixel vicini
(i,j)
Media
Mediana
Gaussiano
Riduce rumore
Media Pesata
pixel centrale ha maggior peso
pixel lontani dal centro hanno un peso minore
Derivata prima
Operatori locali differenziabili
Sviluppo di Taylor
Sobel (I° ordine)
Filtro di Sharpening
Approssimazione di ∇I
Derivata seconda
Usato per rilevare i bordi in una immagine
Laplaciano (II° ordine)
Zero-crossing dell'immagine: deve cambiare l'intensità dei pixel

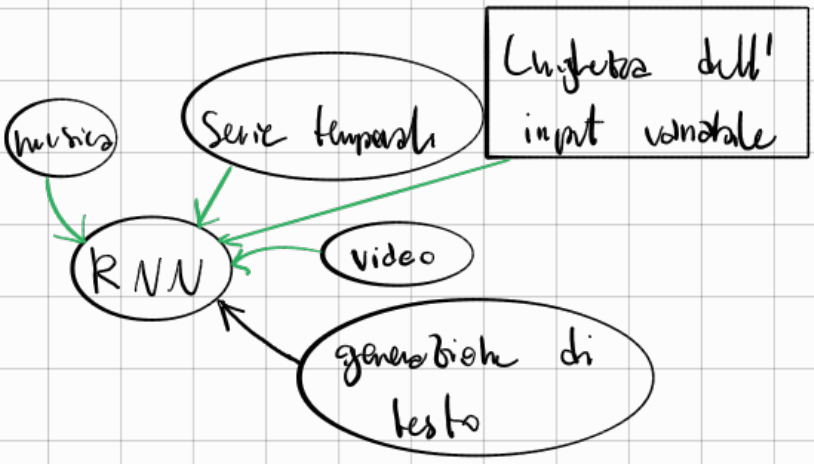
Operazione convoluzione: $(x * w) = \int_{\mathbb{R}} x(a)\, w(t-a)\, da$ $N=1$

Proprietà: commutativa, lineare

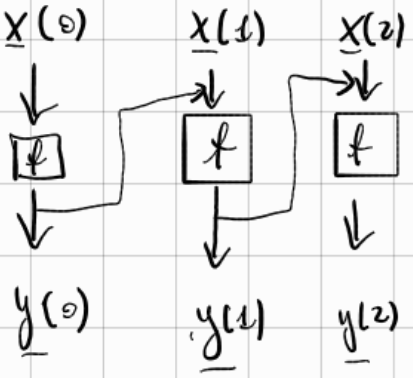
invariante rispetto al tempo ($N=1$) e allo spazio ($N=2$) $(X * W)(u,v) = \int_{\mathbb{R}^2} X(a,b)\, W(u-a, v-b)\, da\, db$

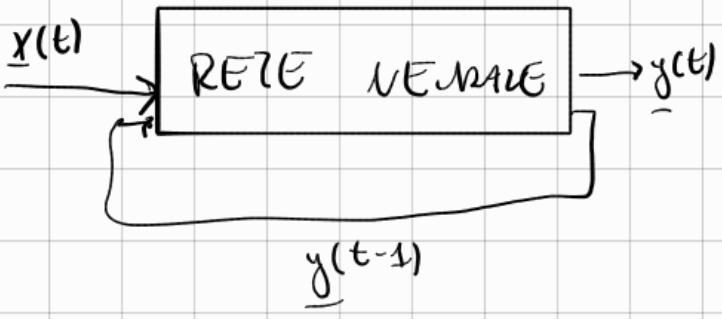






Architettura RNN:





$$\sigma\left(W \cdot \underline{x}(t) + \hat{W} \cdot \underline{y}(t-1) + \underline{b}\right)$$

Segnale corrente — Segnale prec.

Fase di backpropagation

Vanishing gradient

↳ valori troppo bassi

la rete smette di imparare

Soluzioni: Funz. di attivazione modern (ReLU)

Batch Normalization

Inizializzazione dei pesi

Exploding gradient

↳ valori troppo alti

la rete invece di convergere, diverge

Soluzioni: gradient clipping

Inizializzazione dei pesi

Reti generative

NN (Neural Network) → approssimazione deterministica di una funzione f

$\underline{x} \xmapsto{f} \underline{y}$

$\mathbb{R}^N \to \mathbb{R}^M$

Rete generativa → approssimazione stocastica, impara una distribuzione di probabilità

$\underline{x} \xmapsto[NN]{f} \underline{z} \xmapsto[NN]{g} \underline{y}$ (approx.)

$g(\underline{z}, \theta_g)$ — parametri

Reti avversarie generative (GAN)

Si addestrano 2 reti in simultanea

G: Rete generatore: $g(\underline{z}, \theta_g)$ $\left[\underline{x} \xmapsto{G} \underline{z} \mapsto \underline{y}\right]$

D: Rete discriminatore: Rete deterministica (MLP, CNN)

$\underline{x} \mapsto d(x, \theta_x)$ CLASSIFICA L'OUTPUT di G → REAL / FAKE

Per addestrare le due reti si sfrutta un gioco a somma zero

- Il payoff di D è dato da $v(\theta_g, \theta_x)$
- Il payoff di G è dato da $-v(\theta_g, \theta_x)$

Quindi la rete vuole

massimizzare il payoff.

Algoritmo di min max

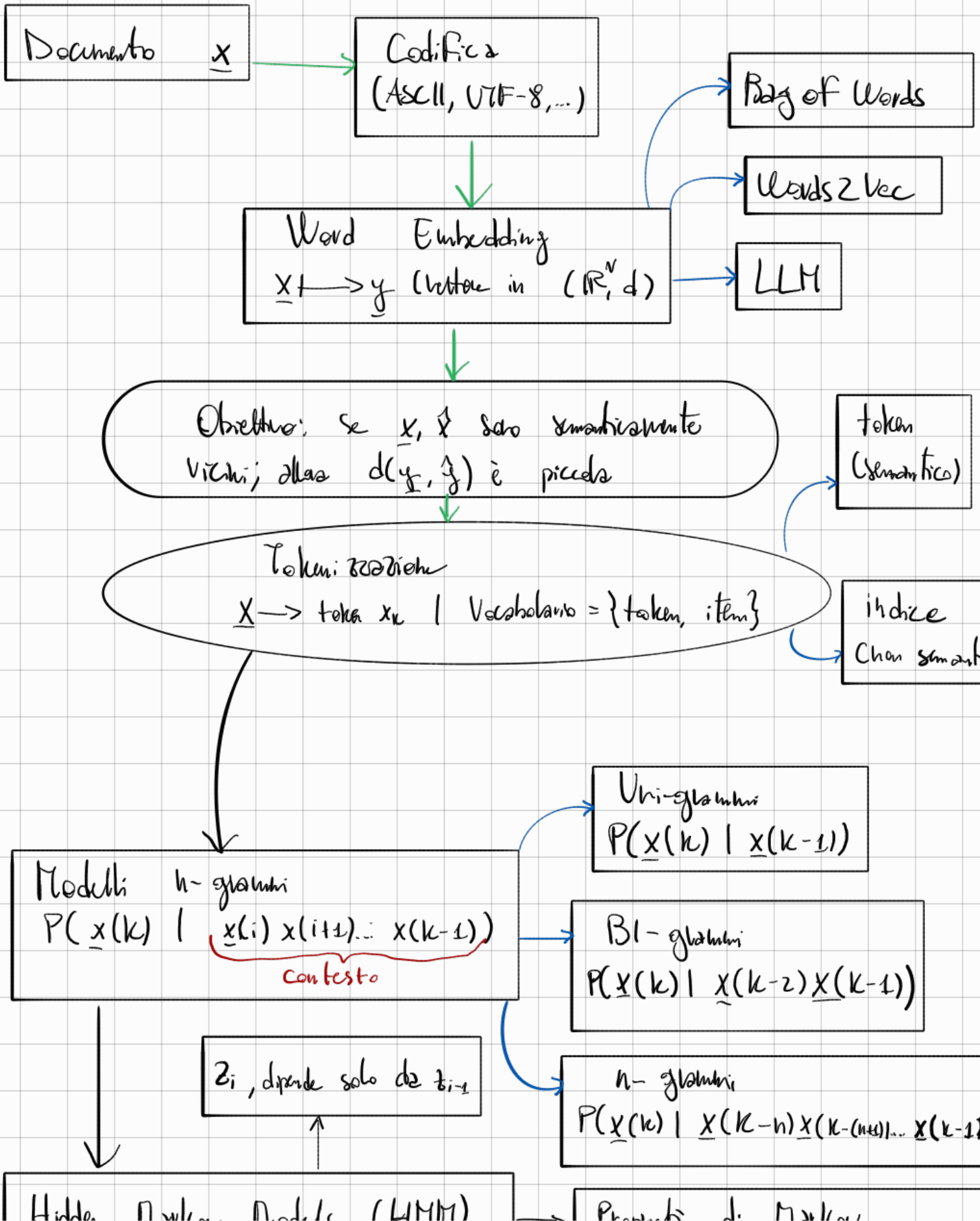
$$g^* = \arg \min_g \max_d v(g, d)$$

(θ_g sotto g, θ_x sotto d)

Se si arriva a convergenza, si ottiene il generatore g^* quando $d(x, \theta_x) \approx \frac{1}{2}$ per ogni x

Non c'è garanzia d'equilibrio

Modelli di linguaggio



Variabili latenti z_k (memoria)

$$z_k = \sigma(W^T x_{k-1} + \hat{W}^T z_{k-1} + b)$$

$P(x(k) \mid x(0), \dots x(k-1)) = P(x(k) \mid x(k-1))$

Obiettivo addestramento:

Minimizzare cross-entropy ⟶

→ Max la likelihood ($\mathcal{L}(\theta, \underline{x})$)

Cross-entropy: funzione di loss, misura la discrepanza tra la distribuzione vera di dati e quella predetta dal modello

Funzione alt. softmax:

$$\sigma(x_i) = \frac{e^{x_i}}{\sum_{k=0}^{n} e^{x_k}}$$

normalizza l'output in una distribuzione di probabilità